

Smart Task Manger

INHALTSVERZEICHNIS

1. Einleitung
 - 1.1 Projektbeschreibung
 - 1.2 Projektziel
 - 1.3 Verwendete Technologien
2. Requirements Engineering
 - 2.1 Anforderungen
 - 2.2 Umsetzung in Notion und Jira
 - 2.3 Screenshots und Nachweise
3. UML-Diagramme
 - 3.1 Verwendetes Werkzeug
 - 3.2 Übersicht der Diagramme
 - 3.3 Reflexion
4. Vorgehensmodell
 - 4.1 Vergleich Scrum und Kanban
 - 4.2 Entscheidung für Kanban
 - 4.3 Kanban Board und Tasks
5. AI / Vibe Coding
 - 5.1 Verwendete Werkzeuge
 - 5.2 Prompts und Ergebnisse
 - 5.3 Reflexion
6. Refactoring
 - 6.1 Ausgewählte Refactorings
 - 6.2 Beispiel ohne AI
 - 6.3 Beispiel mit AI
7. Buildmanagement und CI/CD
 - 7.1 Build-Werkzeug
 - 7.2 GitHub Actions / CI-CD
 - 7.3 Ergebnis

- 8. Testing und Qualität
 - 8.1 Teststrategie
 - 8.2 Beispieltests
 - 8.3 Qualitätssicherung
- 9. Fazit
 - 9.1 Zusammenfassung
 - 9.2 Erfahrungen
 - 9.3 Ausblick
- 10. Anhang
 - 10.1 Screenshots
 - 10.2 Diagramme
 - 10.3 Codebeispiele
 - 10.4 Prompts

1. Einleitung

1.1 Projektbeschreibung

In dieser Arbeit beschäftige ich mich mit meinem Projekt **Smart Task Manager**. Dabei handelt es sich um eine einfache Java-Konsolenanwendung zur Verwaltung von Aufgaben. Die Idee des Projekts ist, dass ein Benutzer seine Aufgaben direkt über die Konsole erstellen, anzeigen, löschen, sortieren und als erledigt markieren kann.

Jede Aufgabe besitzt wichtige Informationen wie einen Titel, eine Priorität, einen Status und ein Fälligkeitsdatum. Dadurch soll der Benutzer einen besseren Überblick über offene und erledigte Aufgaben bekommen. Das Projekt ist bewusst nicht zu groß gewählt, damit die einzelnen Themen aus dem Modul Softwaretechnik gut daran angewendet werden können.

Der Smart Task Manager wurde im Laufe des Moduls für mehrere Aufgaben verwendet, zum Beispiel für Requirements Engineering, UML-Diagramme, die Wahl eines Vorgehensmodells, Vibe Coding, Refactoring und später auch Testing. Dadurch entsteht Schritt für Schritt ein kleines, aber nachvollziehbar geplantes Softwareprojekt.

1.2 Projektziel

Das Ziel des Projekts ist es, eine einfache und verständliche Aufgabenverwaltung als Java-Konsolenprogramm zu entwickeln. Der Fokus liegt nicht auf einem modernen Design oder einer grafischen Oberfläche, sondern auf einer klaren Struktur und einem gut nachvollziehbaren Aufbau des Codes.

Mit dem Programm soll der Benutzer grundlegende Aufgaben verwalten können. Dazu gehören das Erstellen und Anzeigen von Aufgaben, das Löschen von Aufgaben, das Markieren als erledigt sowie das Sortieren nach Priorität oder Fälligkeitsdatum.

Für mich ist das Projekt außerdem ein Lernprojekt. Ich kann daran üben, wie man Anforderungen dokumentiert, UML-Diagramme erstellt, ein Vorgehensmodell auswählt, mit einem Kanban-Board plant und später Code testet und verbessert. Deshalb wurde das Projekt in kleinere Schritte beziehungsweise Iterationen aufgeteilt.

1.3 Verwendete Technologien

Für die Umsetzung und Dokumentation des Projekts **Smart Task Manager** wurden verschiedene Werkzeuge verwendet. Jedes Werkzeug hatte dabei eine bestimmte Aufgabe im Projektverlauf:

- **Java:** Programmiersprache für die Umsetzung der Konsolenanwendung
- **Visual Studio Code:** Entwicklungsumgebung für die Programmierung
- **Git / GitHub:** Versionsverwaltung und Speicherung des Projektstands
- **Notion:** Dokumentation und Verwaltung der Requirements
- **Jira:** Kanban-Board, Backlog und Aufgabenplanung
- **Miro:** Brainstorming, Ideensammlung und erste Strukturierung
- **draw.io:** Erstellung der UML-Diagramme
- **Mermaid:** Textbasierte Erstellung und Visualisierung von UML-Diagrammen
- **Codex:** Unterstützung beim Vibe Coding und bei der Code-Erstellung
- **ChatGPT:** Unterstützung bei Formulierungen, Strukturierung und Rechtschreibkorrektur

- **JUnit:** Geplant für spätere Tests wichtiger Funktionen wie Erstellen, Löschen und Sortieren von Aufgaben

2. Requirements Engineering

2.1 Anforderungen

Für das Projekt **Smart Task Manager** wurden die wichtigsten Anforderungen systematisch dokumentiert. Insgesamt wurden **acht Anforderungen** beschrieben. Diese Anforderungen beziehen sich auf die zentralen Funktionen der Anwendung, zum Beispiel das Erstellen, Anzeigen, Sortieren, Löschen und Abschließen von Aufgaben.

Die Anforderungen wurden so formuliert, dass sie möglichst eindeutig und prüfbar sind. Besonders wichtig war dabei, dass jede Anforderung nicht nur einen Namen bekommt, sondern auch mit weiteren Informationen beschrieben wird. Dadurch kann später besser nachvollzogen werden, was genau umgesetzt und getestet werden soll.

Zu den verwendeten Attributen gehören unter anderem **ID, Datum, Autor, Kurzbeschreibung, Beschreibung, Kategorie, Priorität, Status, Abnahmekriterium, Abhängigkeit und Historie**. Diese Attribute helfen dabei, die Anforderungen übersichtlich zu strukturieren und Missverständnisse zu vermeiden.

Ein Beispiel ist die Anforderung „**Aufgabe erstellen**“. Diese beschreibt, dass der Benutzer über die Konsole eine neue Aufgabe anlegen kann. Als Abnahmekriterium gilt, dass die Aufgabe erfolgreich gespeichert und anschließend in der Aufgabenliste angezeigt wird.

2.2 Umsetzung in Notion und Jira

Für die Dokumentation der Anforderungen wurden zwei verschiedene Werkzeuge verwendet: **Notion** und **Jira**.

Als erstes Werkzeug wurde **Notion** genutzt. In Notion wurden die Anforderungen in einer Tabelle dargestellt. Dieses Werkzeug war besonders hilfreich, weil die Anforderungen schnell erstellt und mit verschiedenen Attributen ergänzt werden

konnten. Für ein kleines Projekt wie den Smart Task Manager war Notion sehr praktisch, da die Tabelle übersichtlich bleibt und einfach angepasst werden kann.

Als zweites Werkzeug wurde **Jira** verwendet. Jira wurde genutzt, um die Anforderungen professioneller als Issues beziehungsweise Aufgaben zu verwalten. Dort konnten einzelne Anforderungen einem Status zugeordnet werden, zum Beispiel offen, in Bearbeitung oder abgeschlossen. Außerdem war es möglich, zu einzelnen Anforderungen Subtasks zu ergänzen. Dadurch wurde deutlicher, welche kleineren Arbeitsschritte zu einer Anforderung gehören.

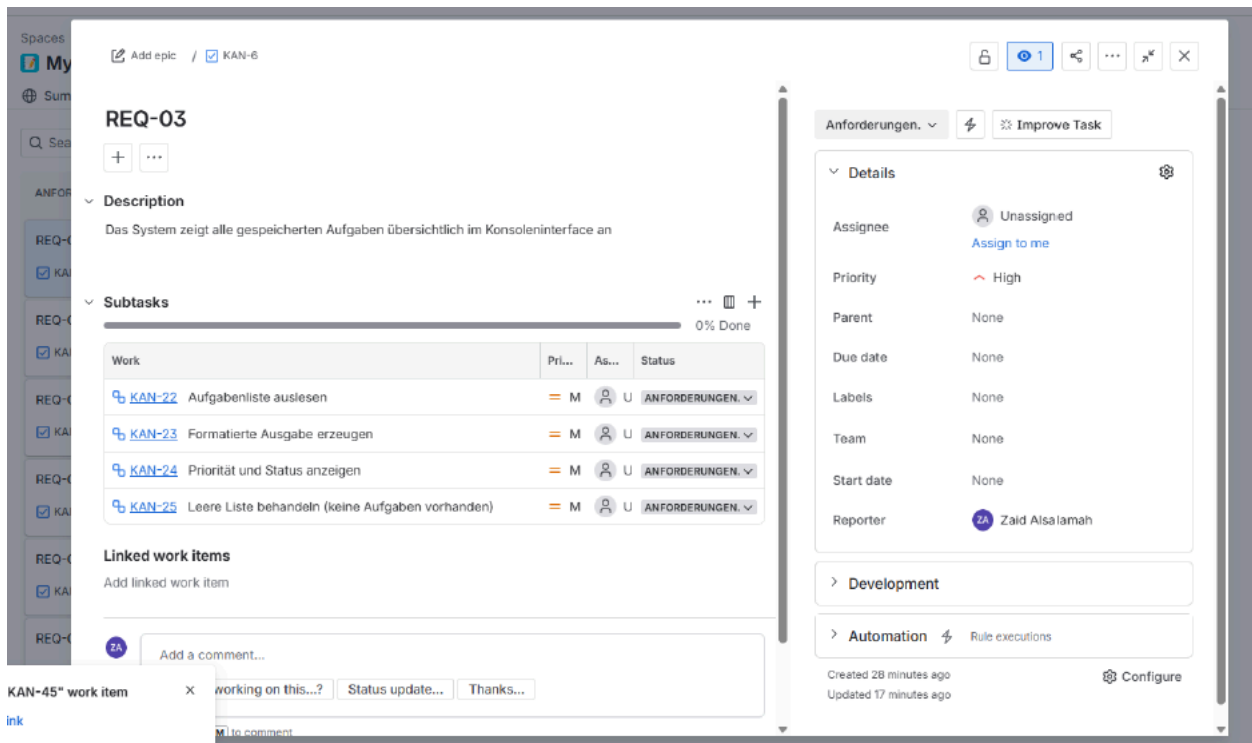
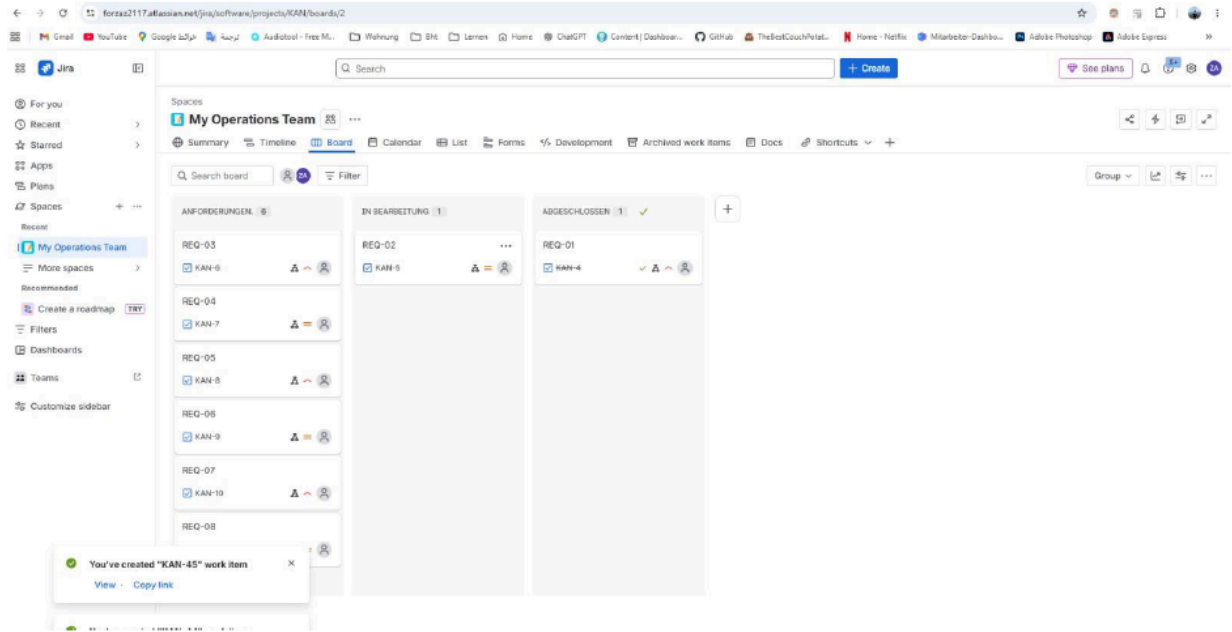
Im Vergleich war Notion einfacher und schneller zu bedienen. Jira war am Anfang etwas komplizierter, bietet aber mehr Möglichkeiten für die spätere Projektplanung und Nachverfolgung. Für mein Projekt war die Kombination aus beiden Werkzeugen sinnvoll: Notion für die einfache Übersicht und Jira für die strukturierte Verwaltung der Aufgaben.

2.3 Screenshots und Nachweise

Werkzeug A (Notion):

ID	Titel	Beschreibung	Priorität	Status	Kategorie	Autor	Version
REQ-01	Benutzer kann Aufgabe erstellen	Der Benutzer kann eine neue Aufgabe erstellen	Hoch	In Bearbeitung	Funktional	Benutzer	Version 1.0 erstellt
REQ-02	Aufgabe löschen	Der Benutzer kann eine bestehende Aufgabe löschen	Mittel	Offen	Funktional	Benutzer	Version 1.0 erstellt
REQ-03	Aufgaben ordnen	Das System zeigt alle gespeicherten Aufgaben nach Priorität	Hoch	Offen	Funktional	Benutzer	Version 1.0 erstellt
REQ-04	Aufgaben priorisieren	Aufgaben können nach Priorität sortiert werden	Mittel	Offen	Funktional	Benutzer	Version 1.0 erstellt
REQ-05	Aufgabe abschließen	Benutzer kann eine Aufgabe als abgeschlossen markieren	Hoch	Offen	Funktional	Benutzer	Version 1.0 erstellt
REQ-06	Aufgaben nach Datum sortieren	Aufgaben werden nach Datum sortiert	Mittel	Offen	Funktional	Benutzer	Version 1.0 erstellt
REQ-07	Systemmeldung	Das System soll schrittweise verbessert werden	Hoch	Offen	Nicht funktionell	Benutzer	Version 1.0 erstellt
REQ-08	Benutzerhandreichheit	Die Benutzeroberfläche soll intuitiv und leicht zu bedienen sein	Mittel	Offen	Nicht funktionell	Benutzer	Version 1.0 erstellt

Werkzeug B (Jira):



Roadmap

```
# Roadmap

## Iteration 1
- Grundstruktur des Projekts erstellen
- Task-Klasse implementieren
- Aufgaben erstellen und anzeigen

## Iteration 2
- Aufgaben löschen
- Aufgaben als erledigt markieren
- Sortierung nach Priorität

## Iteration 3
- Sortierung nach Datum
- Verbesserung der Benutzerführung
- Vorbereitung von Tests (JUnit)
```

Tech-Stack

```
# Tech Stack

## Programmiersprache
- Java
```

```
## Architektur
- Objektorientierte Programmierung (OOP)

## Tools
- Notion (Requirements Management)
- Jira (Professionelles Tool)
- Git / GitHub (Versionierung)

## Testing
- JUnit (Unit Tests)

## Entwicklungsumgebung
- IntelliJ IDEA oder Eclipse
```

3. UML-Diagramme

3.1 Verwendetes Werkzeug

Für die UML-Diagramme wurden **draw.io** und **Mermaid** verwendet. draw.io wurde genutzt, um die Diagramme visuell zu erstellen und manuell anzupassen. Mermaid wurde zusätzlich verwendet, um ein Diagramm textbasiert zu erstellen und im Live-Viewer anzeigen zu lassen.

Für mich war der Unterschied interessant: In draw.io konnte ich die Diagramme direkt zeichnen, während Mermaid mit Code funktioniert. Dadurch konnte ich sehen, dass UML-Diagramme nicht nur grafisch, sondern auch textbasiert erstellt werden können.

3.2 Übersicht der Diagramme

Für das Projekt **Smart Task Manager** wurden mehrere UML-Diagramme erstellt.

Zuerst wurde ein **Klassendiagramm** erstellt.

Das Klassendiagramm beschreibt die grundlegende objektorientierte Struktur des **Smart Task Managers**. Die zentrale Klasse ist `TaskManager`, da sie für die Verwaltung der Aufgaben zuständig ist. Dort befinden sich wichtige Methoden wie `addTask()`, `deleteTask()`, `showTask()`, `sortTask()` und `completeTask()`.

Eine einzelne Aufgabe wird durch die Klasse `Task` dargestellt. Diese Klasse enthält wichtige Informationen wie `id`, `title`, `priority`, `status` und `dueDate`. Zusätzlich besitzt sie Methoden, um den Zustand oder die Priorität einer Aufgabe zu ändern.

Die Klasse `ConsoleUI` übernimmt die Interaktion mit dem Benutzer über die Konsole. Sie zeigt das Menü an und liest Benutzereingaben ein. Damit falsche Eingaben vermieden werden, wird zusätzlich ein `InputValidator` verwendet.

Weitere unterstützende Klassen sind `Storage`, `TaskSorter`, `Priority`, `TaskStatus`, `Category` und `User`. `Storage` ist für das Speichern und Laden von Aufgaben vorgesehen. `TaskSorter` unterstützt die Sortierung nach Priorität oder Fälligkeitsdatum. `Priority` und `TaskStatus` beschreiben feste Werte für Priorität und Zustand einer Aufgabe.

Im Klassendiagramm werden außerdem Beziehungen zwischen den Klassen dargestellt. Ein `TaskManager` verwaltet mehrere Aufgaben. Eine Aufgabe besitzt eine Priorität und einen Status. Zusätzlich kann eine Aufgabe optional einer Kategorie

zugeordnet werden. Dadurch wird die Struktur des Programms übersichtlicher und die Verantwortlichkeiten der einzelnen Klassen werden klarer getrennt.

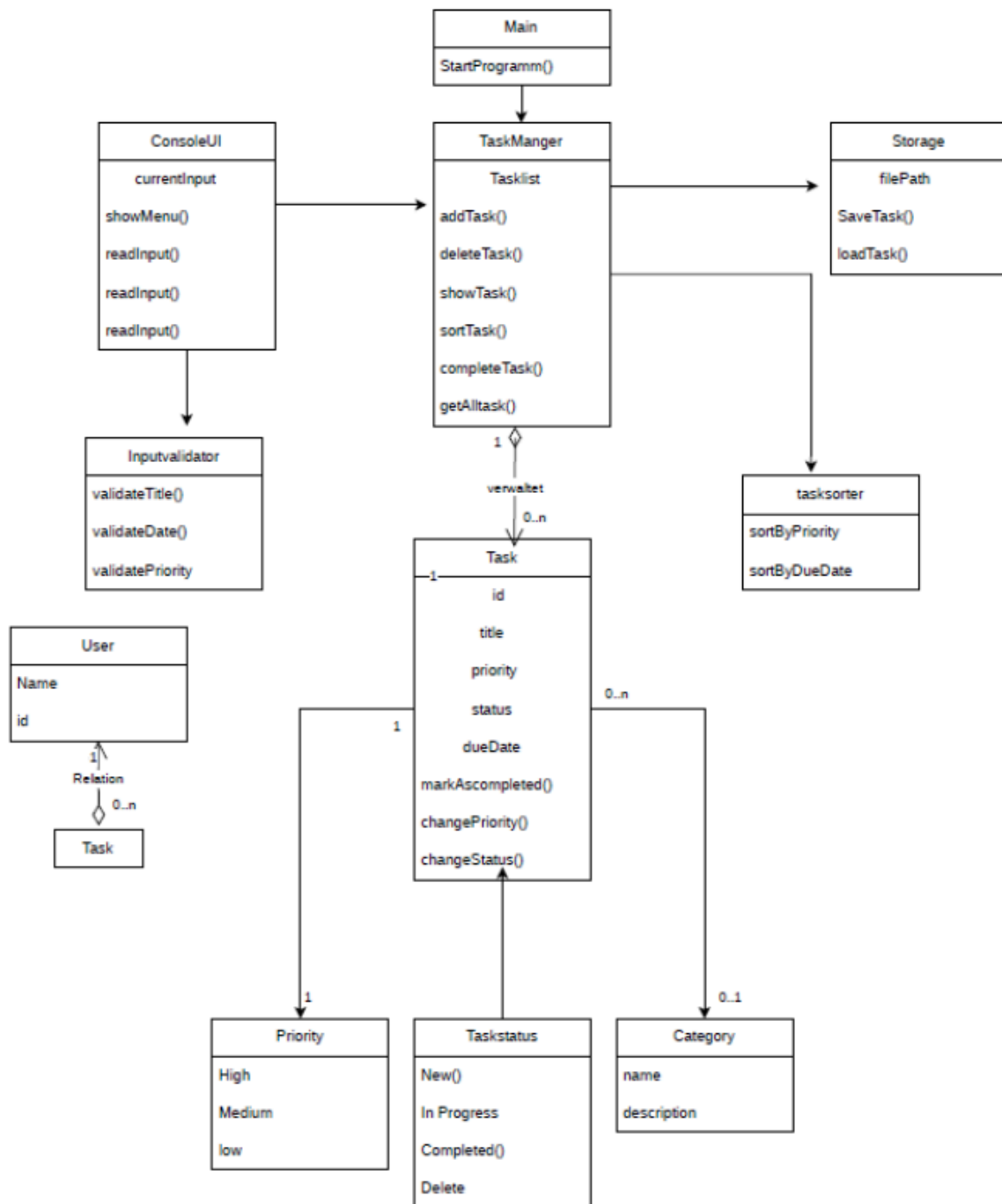


Abbildung: Klassendiagramm des Smart Task Managers.

Danach wurde ein **Use-Case-Diagramm** erstellt. Es zeigt, welche Funktionen der Benutzer im System ausführen kann. Dazu gehören Aufgaben erstellen, Aufgaben anzeigen, Aufgaben löschen, Aufgaben als erledigt markieren und Aufgaben sortieren.

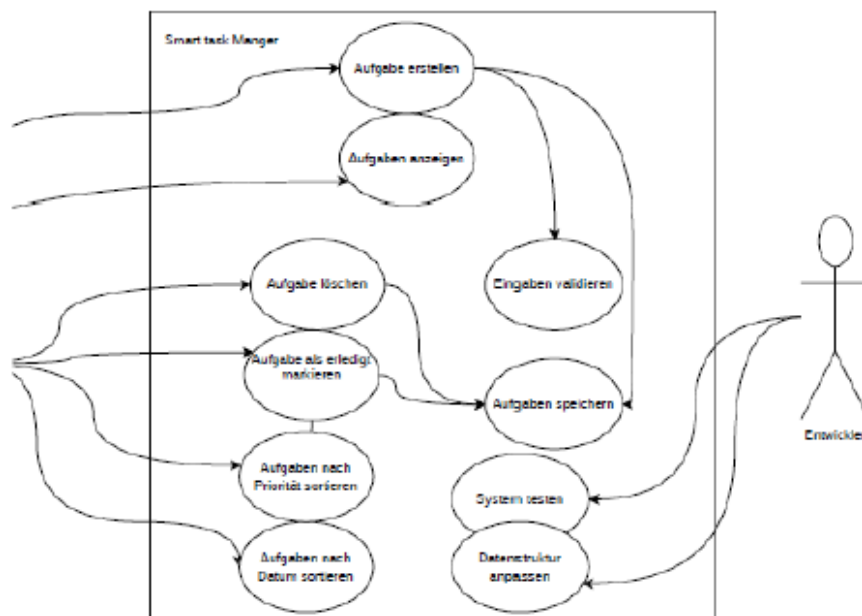
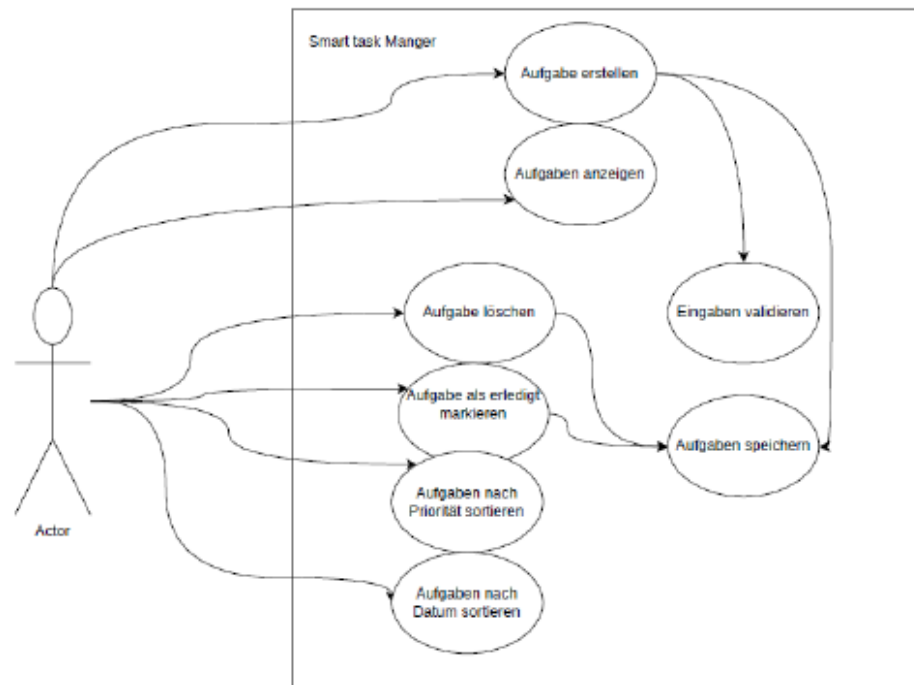


Abbildung: Use-Case-Diagramm mit den wichtigsten Benutzerfunktionen.

Das **Aktivitätsdiagramm** zeigt den Ablauf der Anwendung. Nach dem Start wird das Menü angezeigt. Danach kann der Benutzer verschiedene Aktionen auswählen, zum Beispiel eine Aufgabe erstellen, anzeigen, löschen, abschließen oder sortieren.

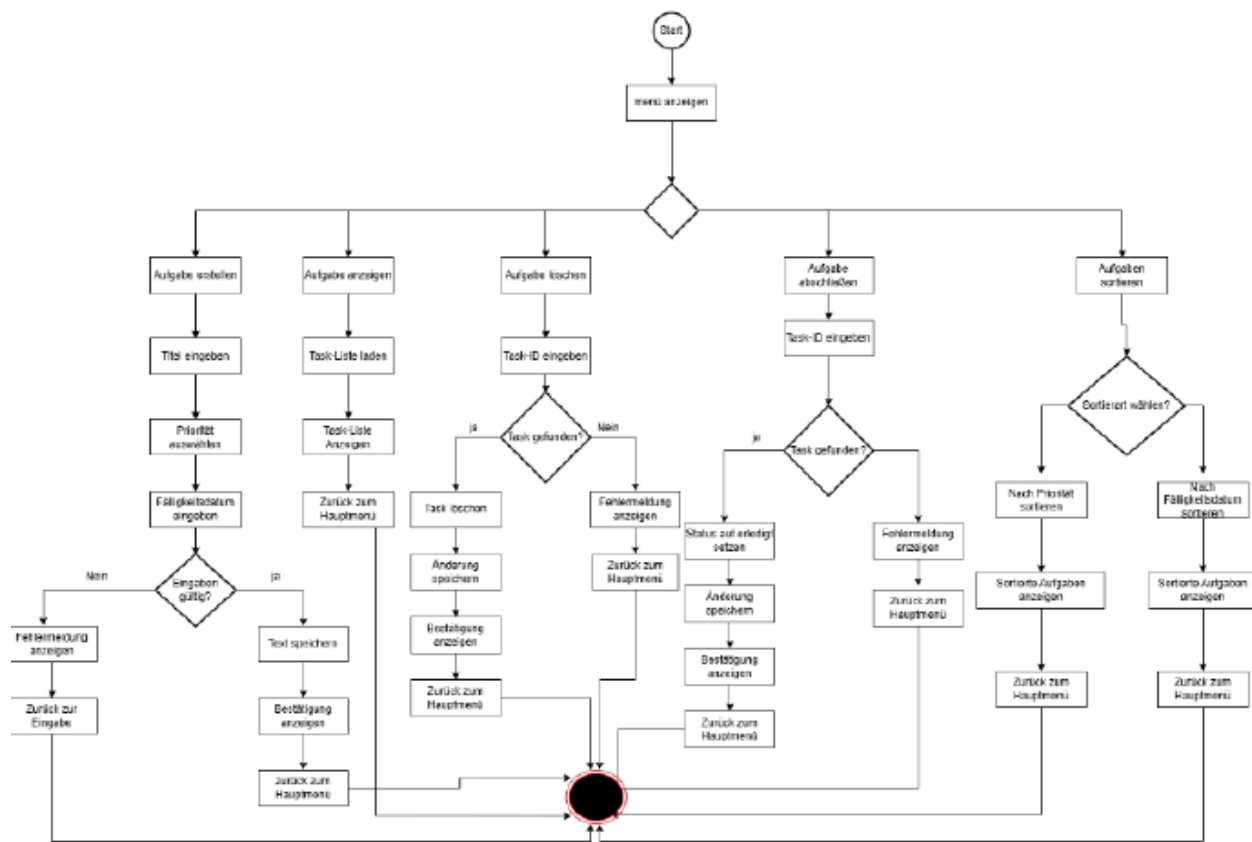


Abbildung: Aktivitätsdiagramm zum Ablauf der Konsolenanwendung.

Das **Sequenzdiagramm** zeigt den Ablauf beim Erstellen und Speichern einer Aufgabe. Dabei wird sichtbar, wie der Benutzer über die `ConsoleUI` Eingaben macht, wie der `InputValidator` die Eingaben prüft und wie der `TaskManager` die Aufgabe speichert.

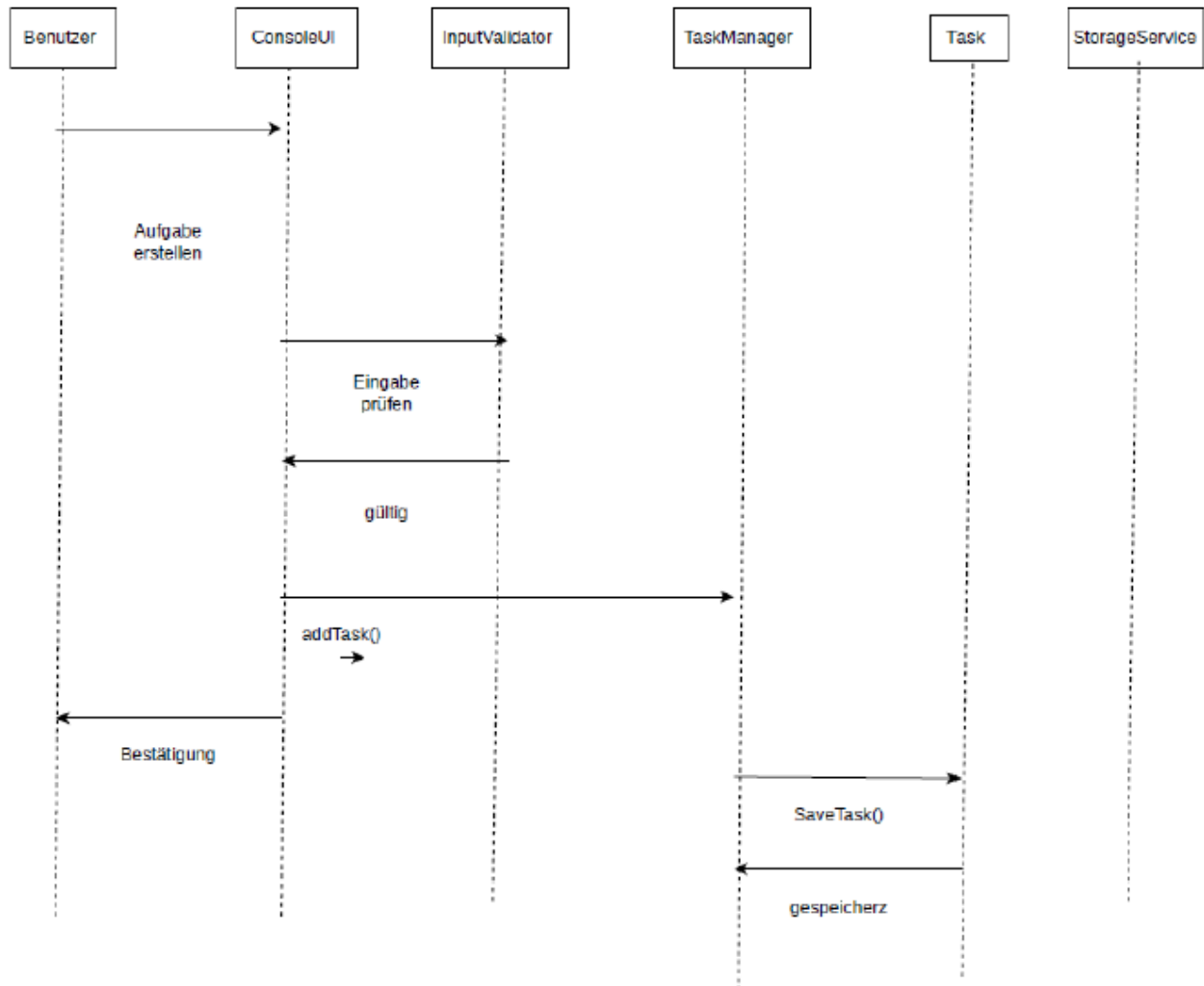


Abbildung: Sequenzdiagramm zum Erstellen und Speichern einer Aufgabe.

Das **Zustandsdiagramm** zeigt den Lebenszyklus einer Aufgabe. Eine Aufgabe kann die Zustände „Neu“, „In Bearbeitung“, „Erledigt“ und „Gelöscht“ haben.

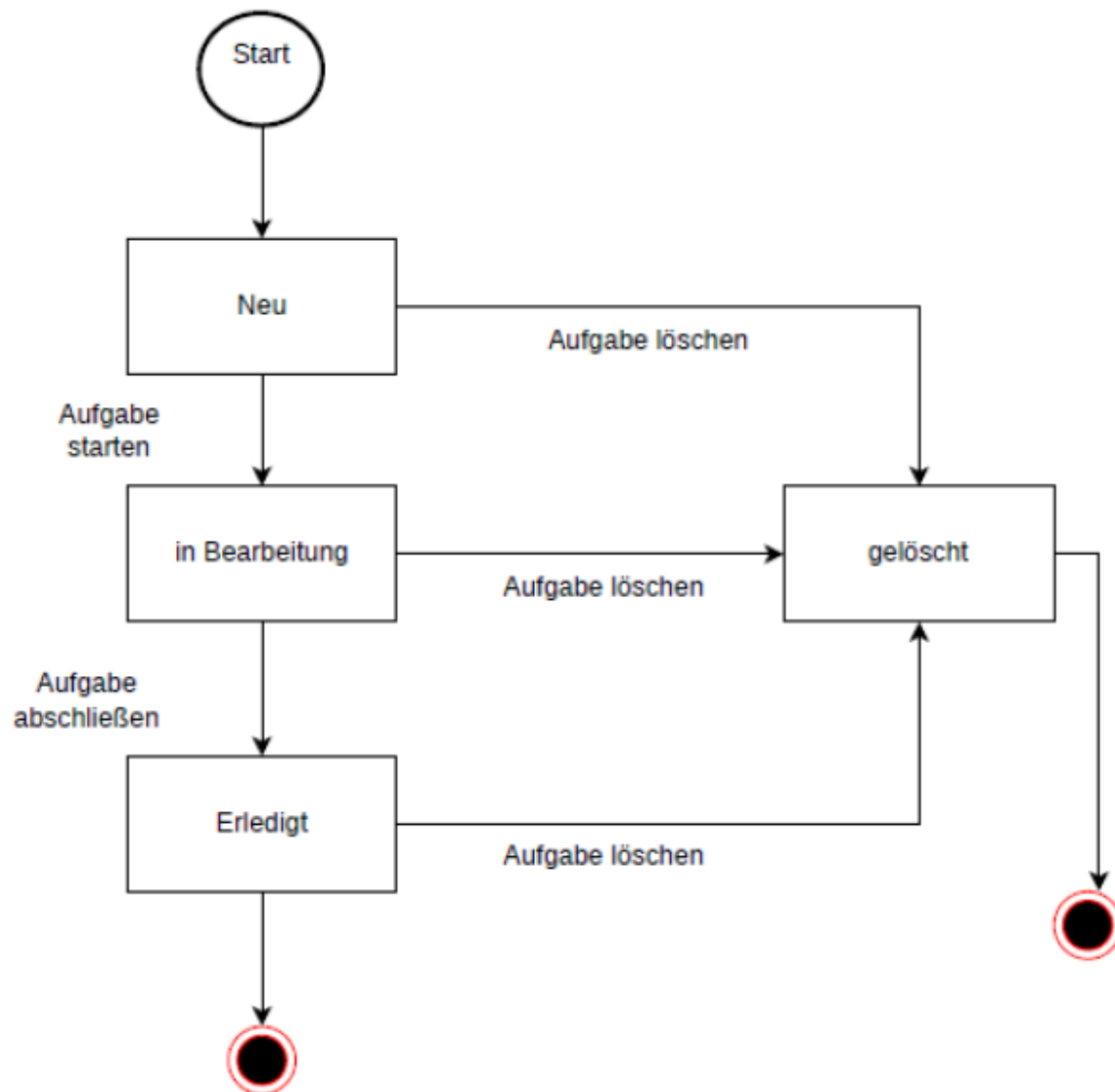


Abbildung: Zustandsdiagramm .

Zusätzlich wurde ein UML-Diagramm mit Hilfe von **Mermaid** und KI textbasiert erstellt. Dafür wurde ein Prompt verwendet, der das Projekt, die gewünschten Diagrammtypen und die Anforderungen an den Mermaid-Code beschreibt.

Code :

```

classDiagram
    class Main {
        startProgram()
    }
    class ConsoleUI {
        showMenu()
        readInput()
        showTasks()
        showMessage()
    }
    class TaskManager {
        taskList
        addTask()
        deleteTask()
        completeTask()
        getAllTasks()
        sortTasks()
    }
    class Task {
        id
        title
        description
        priority
        status
        dueDate
    }
  
```

```

markAsCompleted() changePriority() changeStatus() } class User { id name } class
Priority { HIGH MEDIUM LOW } class TaskStatus { NEW IN_PROGRESS
COMPLETED DELETED } class Category { name description } class
StorageService { filePath saveTasks() loadTasks() } class InputValidator {
validateTitle() validateDate() validatePriority() } class TaskSorter { sortByPriority()
sortByDueDate() } Main → ConsoleUI ConsoleUI → TaskManager ConsoleUI →
InputValidator TaskManager "1" -- "0.." Task : verwaltet TaskManager →
StorageService TaskManager → TaskSorter User "1" → "0.." Task : besitzt Task
"1" → "1" Priority Task "1" → "1" TaskStatus Category "0..1" → "0.." Task :
enthält

```

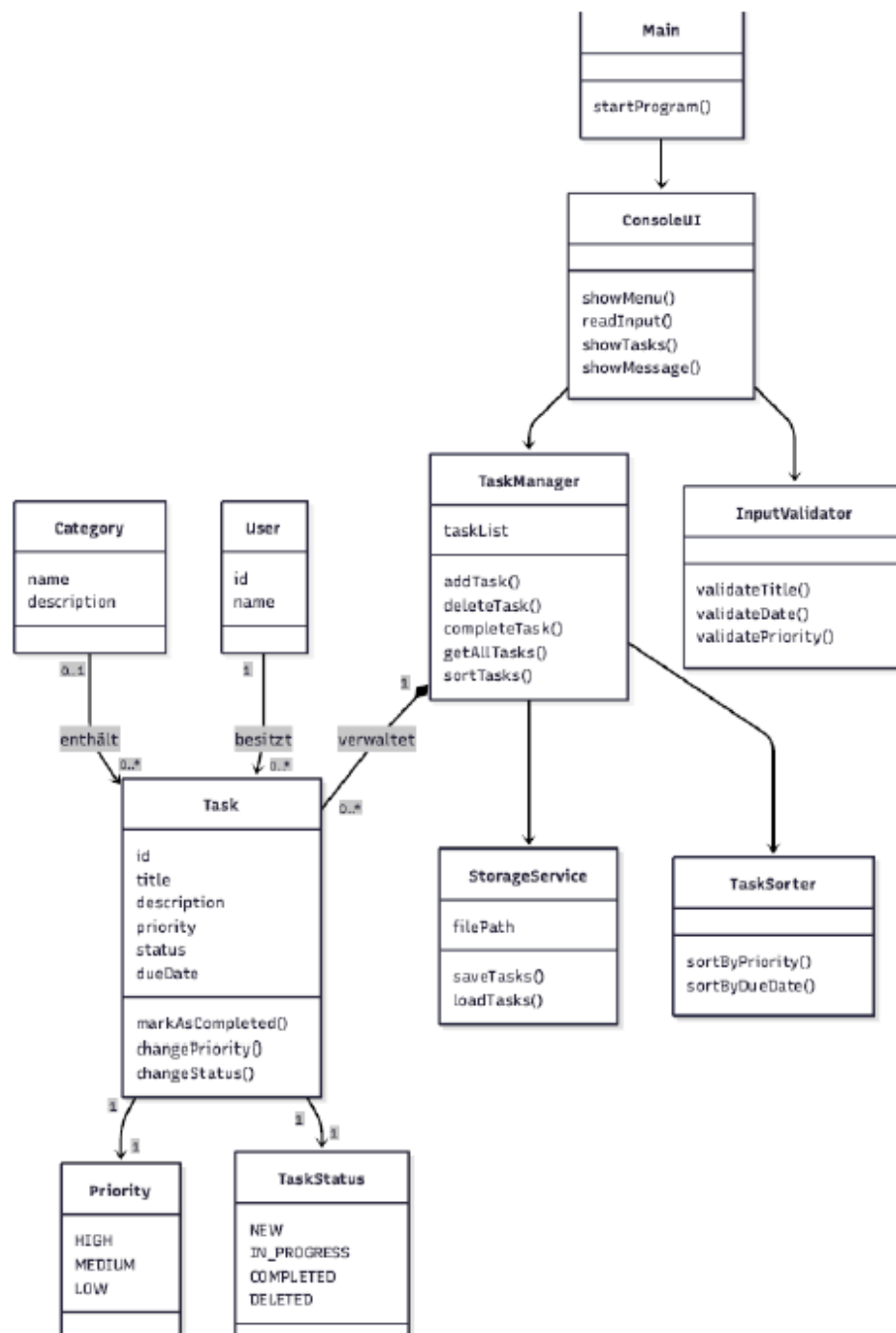


Abbildung: Textbasiertes UML-Diagramm mit Mermaid.

3.3 Reflexion

Beim Erstellen der UML-Diagramme war es für mich nicht immer einfach zu entscheiden, wie detailliert die Diagramme sein sollen. Da der Smart Task Manager ein kleines Java-Konsolenprojekt ist, wollte ich die Diagramme nicht zu kompliziert machen.

Besonders beim Klassendiagramm war die Entscheidung über die Beziehungen zwischen den Klassen schwierig. Zum Beispiel musste ich überlegen, ob zwischen `TaskManager` und `Task` eine einfache Beziehung oder eine Komposition besser passt. Ich habe mich für eine Komposition entschieden, weil der `TaskManager` die Aufgaben verwaltet.

Auch beim Aktivitätsdiagramm war es schwierig, den Ablauf übersichtlich zu zeichnen. Wenn zu viele Rückwege und Aktionen dargestellt werden, wird das Diagramm schnell unklar. Deshalb habe ich versucht, nur die wichtigsten Abläufe zu zeigen.

Bei Mermaid war es ungewohnt, ein Diagramm als Text beziehungsweise Code zu beschreiben. Der Vorteil war aber, dass Änderungen schnell angepasst und direkt angezeigt werden konnten.

4. Vorgehensmodell

4.1 Vergleich Scrum und Kanban

Für das Projekt wurden die Vorgehensmodelle **Scrum** und **Kanban** betrachtet. Beide Modelle gehören zu den agilen Vorgehensweisen, unterscheiden sich aber in der praktischen Anwendung.

Scrum arbeitet mit festen Iterationen, sogenannten Sprints. Außerdem gibt es bestimmte Rollen wie Product Owner, Scrum Master und Entwicklungsteam. Für größere Teams kann Scrum sehr hilfreich sein, weil Aufgaben regelmäßig geplant, überprüft und angepasst werden.

Kanban ist flexibler aufgebaut. Die Arbeit wird auf einem Board sichtbar gemacht und Aufgaben wandern durch verschiedene Spalten, zum Beispiel von **To Do** über **In Progress** bis **Done**. Dadurch sieht man schnell, welche Aufgaben offen sind, welche gerade bearbeitet werden und welche bereits abgeschlossen wurden.

Für mein Projekt war Kanban einfacher verständlich, weil ich keine festen Scrum-Rollen oder komplizierten Meetings gebraucht habe.

4.2 Entscheidung für Kanban

Für den **Smart Task Manager** habe ich mich für **Kanban** entschieden. Das Projekt ist relativ klein und besteht aus mehreren klaren Aufgaben. Deshalb passt ein Kanban-Board gut, um den Fortschritt sichtbar zu machen.

Ein weiterer Grund war, dass ich als einzelner Student an dem Projekt arbeite. Scrum wäre für dieses Projekt etwas zu groß gewesen, weil viele Scrum-Elemente eher für Teams geeignet sind. Kanban hat mir geholfen, meine Aufgaben übersichtlich zu planen und Schritt für Schritt zu bearbeiten.

Obwohl Kanban normalerweise nicht mit festen Sprints arbeitet, wurden die Aufgaben zusätzlich in Iterationen geplant. Dadurch konnte ich die Entwicklung besser strukturieren.

4.3 Kanban Board und Tasks

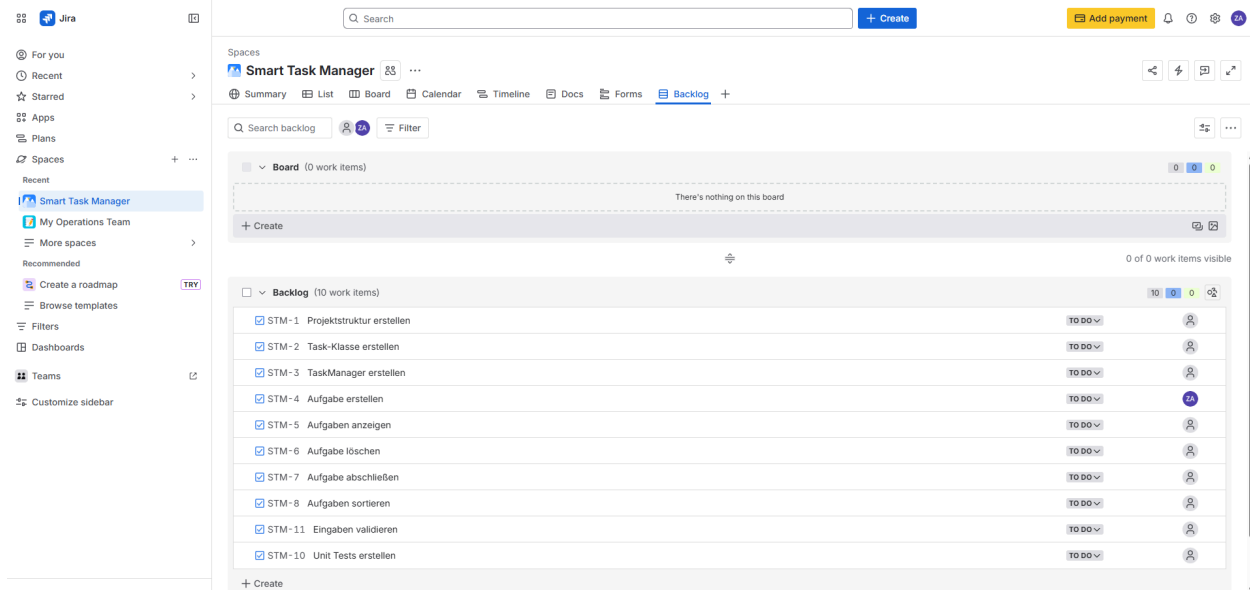
Für die Planung wurde in **Jira** ein Kanban-Board mit dem Namen **Smart Task Manager** erstellt. Das Board enthält die Spalten:

- **To Do**
- **In Progress**
- **Done**

Im Backlog wurden die wichtigsten Aufgaben des Projekts angelegt. Dazu gehören zum Beispiel:

ID	Aufgabe	Priorität
STM-1	Projektstruktur erstellen	Hoch
STM-2	Task-Klasse erstellen	Hoch
STM-3	TaskManager erstellen	Hoch
STM-4	Aufgabe erstellen	Hoch
STM-5	Aufgaben anzeigen	Hoch
STM-6	Aufgabe löschen	Mittel

ID	Aufgabe	Priorität
STM-7	Aufgabe abschließen	Hoch
STM-8	Aufgaben sortieren	Mittel
STM-11	Eingaben validieren	Hoch
STM-10	Unit Tests erstellen	Hoch



Für die erste Iteration wurden vor allem die Grundfunktionen geplant. Dabei waren **Projektstruktur erstellen** und **Task-Klasse erstellen** bereits im Status **Done**. Die Aufgabe **TaskManager erstellen** befand sich im Status **In Progress**. Weitere Aufgaben wie **Aufgabe erstellen**, **Aufgaben anzeigen** und **Eingaben validieren** waren noch im Status **To Do**.

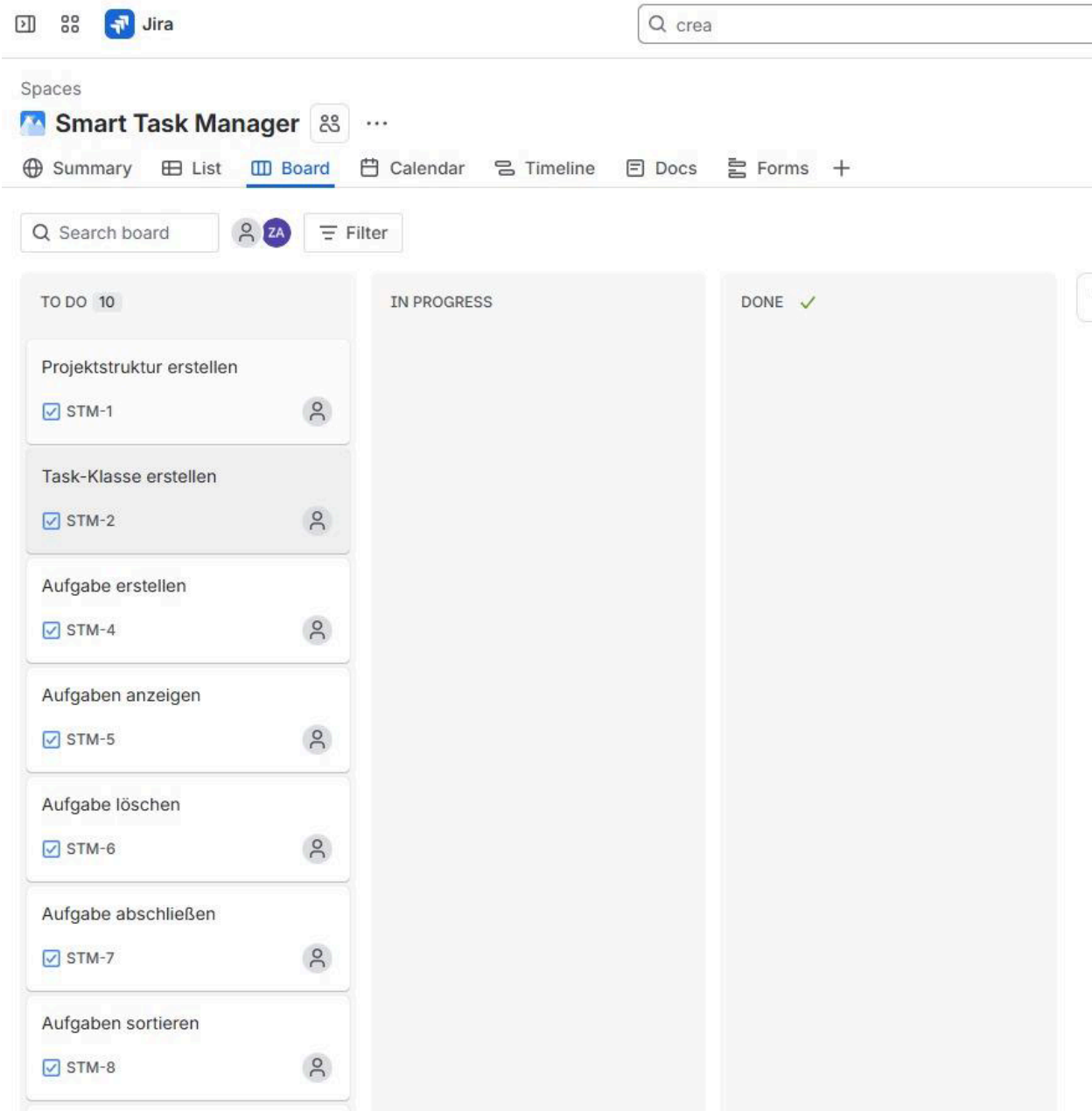


Abbildung: Kanban-Board des Projekts Smart Task Manager..

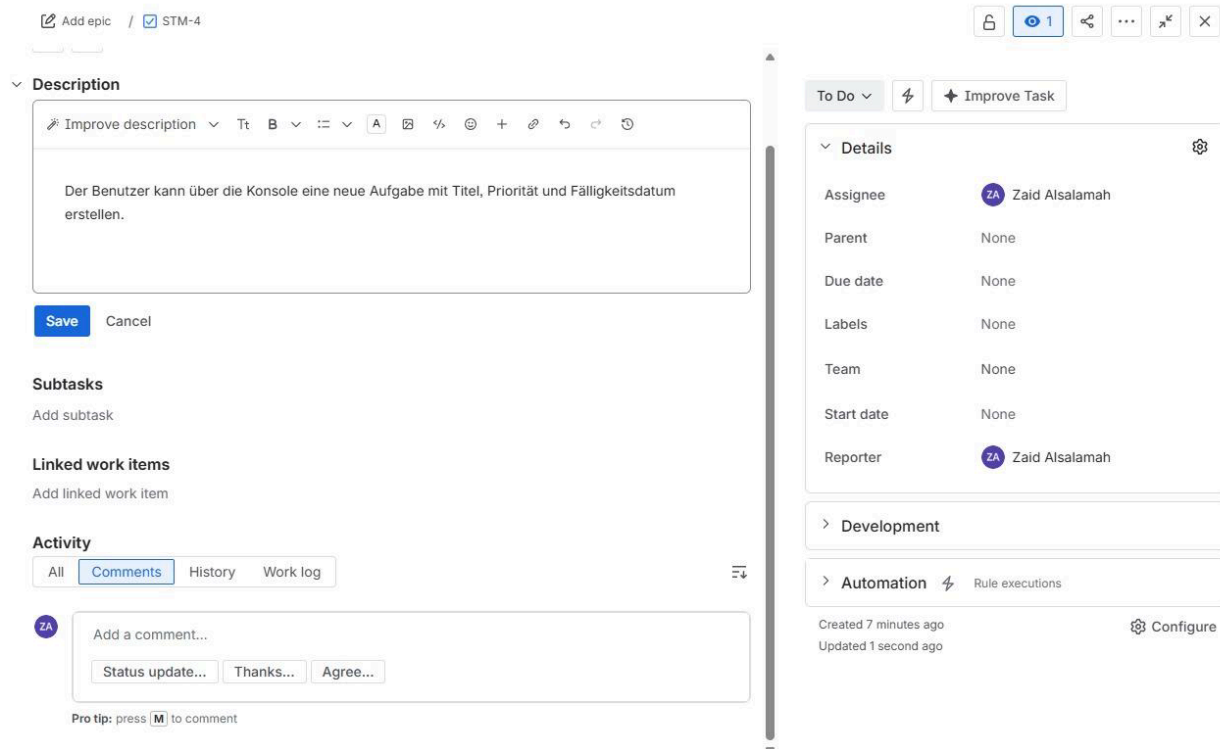


Abbildung: Beispiel eines Jira-Tickets mit Beschreibung und Status.

Durch das Kanban-Board konnte ich gut erkennen, welche Aufgaben bereits erledigt sind und welche noch bearbeitet werden müssen. Für mein Projekt war das sehr hilfreich, weil der Fortschritt einfach sichtbar wurde.

5.1 Verwendete Werkzeuge

Für den Teil **AI / Vibe Coding** wurde hauptsächlich **Codex** verwendet. Codex diente als Unterstützung bei der Erstellung und Verbesserung des Java-Codes für den **Smart Task Manager**. Dabei wurden die Prompts Schritt für Schritt formuliert, damit die Umsetzung zum geplanten Backlog und zu den Anforderungen des Projekts passt.

Zusätzlich wurde **ChatGPT** verwendet, um Prompts besser zu strukturieren, Formulierungen zu verbessern und die Dokumentation sprachlich zu überarbeiten. Die fachlichen Entscheidungen, zum Beispiel dass das Projekt eine einfache Java-Konsolenanwendung ohne Maven, GUI oder Datenbank bleibt, wurden bewusst selbst getroffen.

Als Entwicklungsumgebung wurde **Visual Studio Code** genutzt. Das Projekt wurde mit dem installierten **Java JDK** umgesetzt und direkt über das Terminal kompiliert und gestartet. Für die Versionsverwaltung wurde **GitHub** verwendet, damit der Projektstand gespeichert und nachvollziehbar dokumentiert werden kann.

5.2 Prompts und Ergebnisse

Allgemeine Prompt :

dont start writing any code , just take a look what the Projekt is about . Allgemeiner Projektkontext für alle Prompts

Das Projekt heißt Smart Task Manager.
Es handelt sich um eine einfache Java-Konsolenanwendung zur Verwaltung von Aufgaben.

Die Anwendung soll folgende Funktionen unterstützen:

- Aufgaben erstellen
- Aufgaben anzeigen
- Aufgaben löschen
- Aufgaben als erledigt markieren
- Aufgaben nach Priorität oder Fälligkeitsdatum sortieren
- Eingaben validieren
- wichtige Funktionen mit JUnit testen

Wichtige Vorgaben:

Verwende Java....

Show more ▾

12:07 AM 

Worked for 36s >

Der Projektkontext ist klar: eine einfache, objektorientierte Java-Konsolenanwendung mit Aufgabenverwaltung, Sortierung, Validierung und JUnit-Tests.

Der aktuelle Projektordner ist allerdings vollständig leer. Es gibt noch keine Java-Dateien, Projektstruktur oder Jira-Dokumentation zu STM-1 bis STM-11. Ich habe nichts erstellt oder verändert.

1.Prompt :

Create a beginner-friendly Java console application called "Smart Task Manager".

Important:

- Do not use Maven.
- Do not use Gradle.
- Do not use any external libraries.
- Use only plain Java and the installed JDK.
- The project should be compilable with javac.
- The program should run with java from the terminal.
- Do not create a GUI.
- Do not use a database.
- Keep the code simple and suitable for a beginner student.

Project idea:

The Smart Task Manager is a small console application for managing tasks.

The user should be able to create tasks, display tasks, delete tasks, mark tasks as completed, and sort tasks by priority or due date.

Create this simple project structure:

```
SmartTaskManager/  
├── src/  
│   ├── Main.java  
│   ├── Task.java  
│   ├── TaskManager.java  
│   ├── ConsoleUI.java  
│   ├── Priority.java  
│   ├── TaskStatus.java  
│   └── InputValidator.java  
└── README.md
```

Implementation requirements:

- Main.java starts the application.
- ConsoleUI.java handles the console menu and user input.

- Task.java represents a task with id, title, description, priority, status, and dueDate.
- Priority.java is an enum with HIGH, MEDIUM, LOW.
- TaskStatus.java is an enum with NEW, IN_PROGRESS, COMPLETED, DELETED.
- TaskManager.java manages a list of tasks using ArrayList.
- InputValidator.java validates title, priority, and date input.

Features:

1. Create task
2. Show all tasks
3. Delete task by ID
4. Mark task as completed
5. Show tasks sorted by priority
6. Show tasks sorted by due date
7. Exit program

Use LocalDate for dueDate.

Use simple error messages for invalid input.

Keep the code readable and object-oriented.

Also create a README.md that explains:

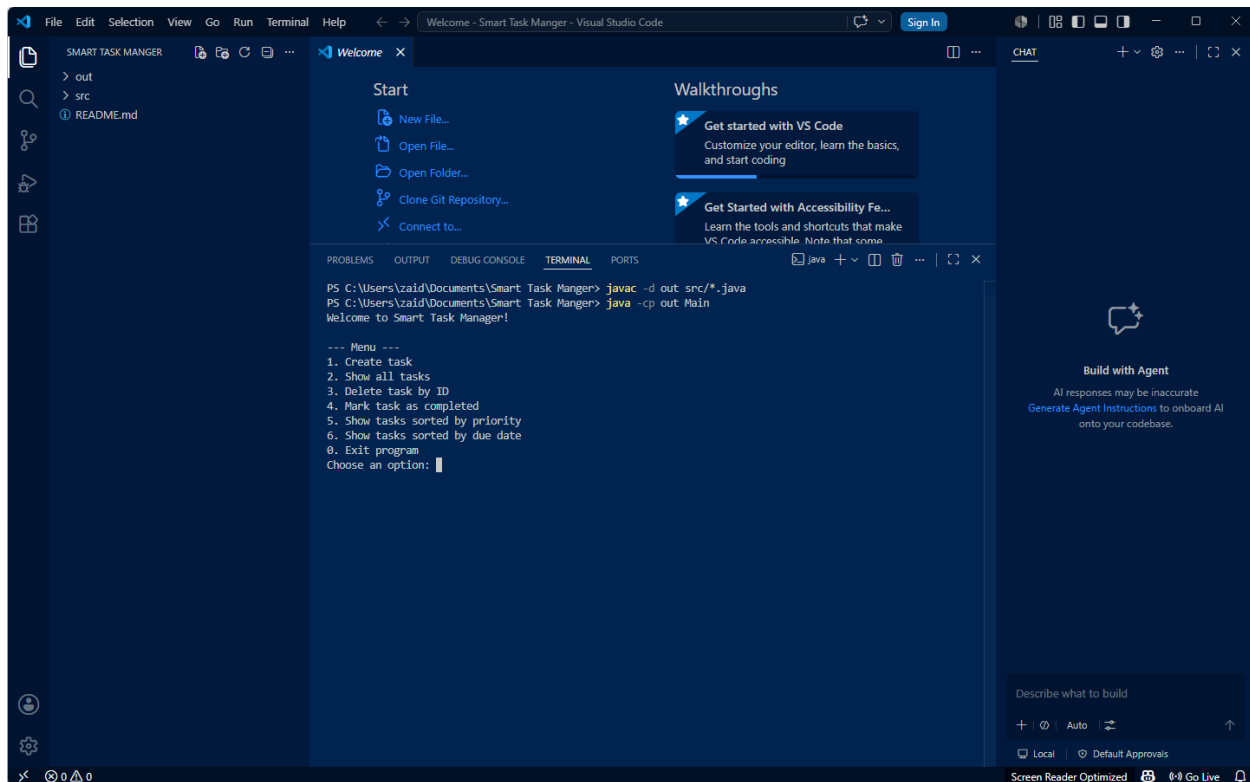
- what the project does
- how to compile it with javac
- how to run it with java

Use these commands in the README:

```
javac -d out src/*.java
```

```
java -cp out Main
```

Nachweis 1.Prompt



2.Prompt :

Review the current Smart Task Manager Java project.

Context:

The project is a simple Java console application without Maven or Gradle.

It should compile with:

```
javac -d out src/*.java
```

And run with:

```
java -cp out Main
```

Please check the existing code for:

- compile errors
- missing imports
- wrong class names
- wrong method calls
- beginner-unfriendly complexity

- problems in the console menu

Important:

- Do not rewrite the whole project.
- Do not add Maven, Gradle, GUI, database, or external libraries.
- Keep the project simple and beginner-friendly.
- Only fix necessary problems.

After fixing, briefly explain what was changed.

2.Prompts result

Worked for 34s >

Review Result

No necessary problems were found, so no source files were changed.

Checked successfully:

- Compilation with `javac -d out src/*.java`
- Correct imports, classes, and method calls
- All menu options
- Invalid numbers, titles, priorities, and dates
- Priority and due-date sorting
- Completing and deleting tasks

The code remains simple, readable, and beginner-friendly.



3.Prompt :

Improve the input validation in the Smart Task Manager project.

Context:

The project is a plain Java console application without Maven.

It uses:

- ConsoleUI for user interaction
- TaskManager for task logic

- Task for task data
- InputValidator for validation

Requirements:

- Title must not be empty.
- Description may be empty, but should not crash the program.
- Priority must be HIGH, MEDIUM, or LOW.
- Due date must be in the format yyyy-MM-dd.
- Invalid dates should not crash the program.
- Invalid menu input should show an error message.
- Invalid task IDs should be handled safely.

Important:

- Do not add GUI.
- Do not add database.
- Do not use external libraries.
- Keep the code simple and readable.
- Do not rewrite the whole project if not necessary.

Please update only the necessary files and explain the changes shortly.

3.Prompt result

Worked for 1m 19s >

Updated only:

- `InputValidator.java` : Added strict `yyyy-MM-dd` validation. Impossible dates such as `2026-02-30` are rejected.
- `ConsoleUI.java` : Task IDs must now be positive numbers.

Existing title, priority, description, and menu validation remained unchanged.

Verified successfully with `javac -d out src/*.java` and invalid console inputs.

📄 🌟 🔍 ↺ 12:50 AM

4.Prompt :

Improve the console output of the Smart Task Manager.

Context:

The project is a beginner-friendly Java console application.

The user can create, show, delete, complete, and sort tasks.

Please improve only the console output.

Requirements:

- The main menu should be easy to read.
- Task output should clearly show:
 - ID
 - title
 - description
 - priority
 - status
 - due date
- Show a helpful message when no tasks exist.
- Show clear success and error messages.
- Keep all text simple and understandable.

Important:

- Do not change the core logic.
- Do not add GUI or database.
- Do not use external libraries.
- Keep the project compilable with:
`javac -d out src/*.java`

Briefly explain what you improved.

4.Prompt result

Worked for 58s >

Improved only the console output:

- Clearer menu layout and headings
- Each task displayed as a readable field list
- Empty descriptions shown as "No description"
- More helpful empty-list message
- Consistent success and error messages with task IDs

Core logic was unchanged. Compilation and console flow were verified successfully.

    12:56 AM

5.Prompt :

Create a simple manual test class for the Smart Task Manager project without JUnit.

Context:

The project does not use Maven or Gradle.

Use only plain Java and the JDK.

Create a file:

src/TaskManagerManualTest.java

The test class should use a main method and test the TaskManager functionality manually.

Test:

- addTask
- deleteTask with existing ID
- deleteTask with non-existing ID
- completeTask
- sorting by priority
- sorting by due date

Requirements:

- Print clear messages like "PASS" or "FAIL".
- Do not use JUnit.
- Do not use external libraries.
- Keep the test code simple and understandable.
- The project should still compile with:
`javac -d out src/*.java`

Please provide the full TaskManagerManualTest.java file.

5.Prompt Result

Worked for 55s >

Created the complete [TaskManagerManualTest.java](#).

It tests all six requested cases using simple `PASS / FAIL` output. Compilation succeeded, and all tests passed:

 Add to chat



Passed: 6

Failed: 0

Run with:

 Add to chat



```
javac -d out src/*.java
java -cp out TaskManagerManualTest
```

    12:59 AM

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
powershell + - [] [X] ... | [X] X

PS C:\Users\zaid\Documents\Smart Task Manger> javac -d out src/*.java
PS C:\Users\zaid\Documents\Smart Task Manger> java -cp out Main
Welcome to Smart Task Manager!

--- Menu ---
1. Create task
2. Show all tasks
3. Delete task by ID
4. Mark task as completed
5. Show tasks sorted by priority
6. Show tasks sorted by due date
0. Exit program
Choose an option: Exception in thread "main" java.util.NoSuchElementException: No line found
    at java.base/java.util.Scanner.nextLine(Scanner.java:1660)
    at ConsoleUI.readNumber(ConsoleUI.java:133)
    at ConsoleUI.start(ConsoleUI.java:21)
    at Main.main(Main.java:5)
PS C:\Users\zaid\Documents\Smart Task Manger> ^C
PS C:\Users\zaid\Documents\Smart Task Manger> javac -d out src/*.java
PS C:\Users\zaid\Documents\Smart Task Manger> javac -d out src/*.java
PS C:\Users\zaid\Documents\Smart Task Manger> javac -d out src/*.java
PS C:\Users\zaid\Documents\Smart Task Manger> java -cp out TaskManagerManualTest
=== TaskManager Manual Tests ===
PASS: Add task
PASS: Delete task with existing ID
PASS: Delete task with non-existing ID
PASS: Complete task
PASS: Sort tasks by priority
PASS: Sort tasks by due date

Passed: 6
Failed: 0
PS C:\Users\zaid\Documents\Smart Task Manger> |
```

6.Prompt

Update the README.md for the Smart Task Manager project.

The project is a simple Java console application without Maven or Gradle.

The README should include:

1. Project name
2. Short project description
3. Features
4. Project structure
5. How to compile the project
6. How to run the project

7. How to run the manual test class

8. Technologies used

9. Short note that the project is part of a software engineering learning project

Use these commands:

Compile:

```
javac -d out src/*.java
```

Run application:

```
java -cp out Main
```

Run manual tests:

```
java -cp out TaskManagerManualTest
```

Keep the README simple, clear, and suitable for a beginner project.

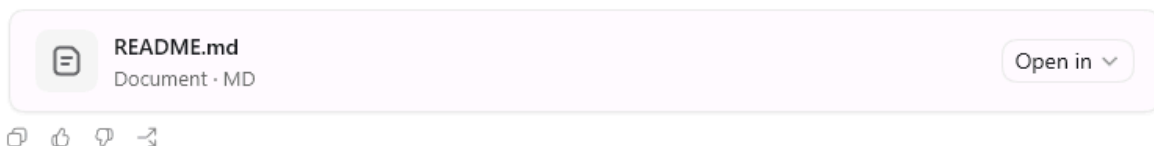
6.Prompt Result

Worked for 32s >

Updated  [README.md](#) with:

- Project description and features
- Current project structure
- Compile and run instructions
- Manual test instructions
- Technologies used
- Software engineering learning-project note

All requested commands are included exactly.



7.Prompt

Review the final Smart Task Manager Java console application.

Project context:

- Java console application
- No Maven
- No Gradle
- No GUI
- No database
- Beginner-friendly code
- Object-oriented structure

Features:

- Create task
- Show tasks
- Delete task
- Mark task as completed
- Sort tasks by priority
- Sort tasks by due date
- Validate input
- Manual test class exists

Please check:

- Does the code match the requirements?
- Is the code understandable for a beginner?
- Are responsibilities separated well between Task, TaskManager, ConsoleUI, and InputValidator?
- Are there unnecessary complexities?
- Are there possible small improvements?

Important:

- Do not rewrite the whole project.
- Only provide a short review and small improvement suggestions.

7.prompt result

Worked for 28s >

Review

The project matches all stated requirements. It compiles successfully, and all six manual tests pass.

Responsibilities are separated clearly:

- `Task` : stores task data
- `TaskManager` : manages and sorts tasks
- `ConsoleUI` : handles interaction and output
- `InputValidator` : validates and converts input

The code is readable, appropriately object-oriented, and not unnecessarily complex.

Small optional improvements:

- `IN_PROGRESS` currently exists but is never used.
- Manual sorting tests could check list size before accessing positions.
- `Task.setStatus()` could eventually become specific methods such as `markCompleted()`, but the current approach is simpler for beginners.

No changes are necessary.



5.3 Reflexion

Die Arbeit mit Codex war hilfreich, weil dadurch schnell eine erste lauffähige Codebasis für den **Smart Task Manager** entstanden ist. Besonders praktisch war, dass die Umsetzung Schritt für Schritt erfolgen konnte und die Prompts direkt an mein Backlog angepasst wurden.

Trotzdem hat nicht alles sofort perfekt funktioniert. Einige Prompts mussten genauer formuliert werden, zum Beispiel dass **kein Maven, keine GUI, keine Datenbank** und **keine externen Bibliotheken** verwendet werden sollen. Erst dadurch passte der generierte Code wirklich zu meinem kleinen Java-Konsolenprojekt.

Ich habe gemerkt, dass Vibe Coding nicht bedeutet, einfach einen einzigen Prompt zu schreiben und das fertige Projekt zu bekommen. Man muss die Ergebnisse prüfen, testen und teilweise anpassen. Für mich war besonders

wichtig zu lernen, dass gute Prompts klare Grenzen und konkrete Anforderungen brauchen.

6.0 Refactoring

6.1 Ausgewählte Refactorings

Für diese Aufgabe habe ich mir den Refactoring-Katalog von Martin Fowler angeschaut. Besonders wichtig für meine Beispiele waren **Extract Function** und **Move Function**.

Extract Function bedeutet, dass ein längerer oder unübersichtlicher Codeabschnitt in eine eigene Funktion beziehungsweise Methode ausgelagert wird. Dadurch wird der Code besser lesbar, weil einzelne Schritte klarer voneinander getrennt sind.

Move Function bedeutet, dass eine Funktion oder Methode in eine passendere Klasse verschoben wird. Das ist sinnvoll, wenn eine Methode eigentlich besser zu einer anderen Klasse passt, weil dort die Verantwortung logischer ist.

Nicht ganz verstanden habe ich am Anfang **Remove Middle Man**. Dabei geht es darum, unnötige Weiterleitungen zwischen Klassen zu entfernen. Bei kleinen Projekten wirkt dieses Refactoring für mich noch nicht so wichtig. Ich denke, dass es eher bei größeren Systemen sinnvoll ist.

6.2 Beispiel ohne AI

Für mein eigenes Refactoring ohne AI habe ich die Klasse `ConsoleUI` ausgewählt. Dort befindet sich die Methode `createTask()`. Diese Methode war vor dem Refactoring relativ lang, weil mehrere Aufgaben direkt in einer Methode ausgeführt wurden. Dazu gehörten das Einlesen des Titels, die Prüfung der Eingabe, das Einlesen der Beschreibung, der Priorität und des Fälligkeitsdatums sowie das Erstellen der Aufgabe.

Als Fowler-Refactoring wurde **Extract Function** verwendet. In Java entspricht das dem Refactoring **Extract Method**. Dabei werden einzelne Teile einer längeren Methode in eigene kleinere Methoden ausgelagert.

Vor dem Refactoring enthält `createTask()` mehrere Eingabeschleifen und Validierungen direkt in der Methode. Dadurch war der Ablauf zwar funktional, aber etwas unübersichtlich.

Nach dem Refactoring wurden Teile der Methode in eigene Methoden verschoben, zum Beispiel:

- `readTitle()`
- `readPriority()`
- `readDueDate()`

Dadurch ist die Methode `createTask()` kürzer und besser lesbar geworden. Sie zeigt jetzt nur noch den Hauptablauf: Titel lesen, Beschreibung lesen, Priorität lesen, Fälligkeitsdatum lesen und anschließend die Aufgabe erstellen.

Das Verhalten des Programms wurde dabei nicht verändert. Die gleichen Eingaben werden weiterhin abgefragt, die gleichen Validierungen werden durchgeführt und die Aufgabe wird wie vorher erstellt. Nur die Struktur des Codes wurde verbessert.

Hier Screenshot einfügen: `createTask()` vor dem Refactoring

Abbildung: Methode `createTask()` vor dem manuellen Refactoring.

Hier Screenshot einfügen: `createTask()` nach dem Refactoring

Abbildung: Methode `createTask()` nach dem manuellen Refactoring mit ausgelagerten Methoden.

Das Refactoring hat den Code übersichtlicher gemacht, weil die einzelnen Schritte der Aufgabenerstellung jetzt klarer getrennt sind.

Before Refactoring :

```

private void createTask() {
    Click to collapse the range.
    {
        System.out.print("Title: ");
        title = scanner.nextLine();
        if (!InputValidator.isValidTitle(title)) {
            System.out.println("Error: The title must not be empty.");
        }
        while (!InputValidator.isValidTitle(title));

        System.out.print("Description: ");
        String description = scanner.nextLine().trim();

        Priority priority = null;
        while (priority == null) {
            System.out.print("Priority (HIGH, MEDIUM, LOW): ");
            priority = InputValidator.parsePriority(scanner.nextLine());
            if (priority == null) {
                System.out.println("Error: Enter HIGH, MEDIUM, or LOW.");
            }
        }

        LocalDate dueDate = null;
        while (dueDate == null) {
            System.out.print("Due date (YYYY-MM-DD): ");
            dueDate = InputValidator.parseDate(scanner.nextLine());
            if (dueDate == null) {
                System.out.println("Error: Enter a valid date in yyyy-MM-dd format.");
            }
        }

        Task task = taskManager.createTask(title.trim(), description, priority, dueDate);
        System.out.println("Success: Task " + task.getId() + " was created.");
    }
}

```

After Refactoring :

```

✓ private void createTask() {
    .....String title = readTitle();
    .....
    .....System.out.print("Description: ");
    .....String description = scanner.nextLine().trim();
    .....
    .....Priority priority = readPriority();
    .....LocalDate dueDate = readDueDate();
    .....
    .....Task task = taskManager.createTask(title, description, priority, dueDate);
    .....System.out.println("Success: Task " + task.getId() + " was created.");
    .....}

```

6.3 Beispiel mit AI

Für das Refactoring mit AI wurde Codex verwendet. Codex sollte zuerst eine passende Codestelle auswählen und danach das Refactoring durchführen.

Ausgewählt wurde die Methode `toString()` in der Klasse `Task`. Als Fowler-Refactoring wurde **Move Function** verwendet. Die Konsolenausgabe wurde aus `Task` entfernt und in die Klasse `ConsoleUI` verschoben.

Dadurch ist `Task` nur noch für die Daten einer Aufgabe zuständig, während `ConsoleUI` die Ausgabe übernimmt. Das Verhalten des Programms blieb gleich und die Tests waren erfolgreich.

before Refactoring:

```

7
8  ✨ @Override
9  ... public String toString(){
10     ... String shownDescription = description.isEmpty() ? "No description" : desc
11
12     ... return "ID:....." + id
13     ...         + "\nTitle:....." + title
14     ...         + "\nDescription: " + shownDescription
15     ...         + "\nPriority:...." + priority
16     ...         + "\nStatus:....." + status
17     ...         + "\nDue date:...." + dueDate;
18     ... }

```

prompt :

Please review my existing Smart Task Manager project and choose one suitable non-trivial code section for an AI-assisted refactoring.

Important:

- Do not change any code yet.
- Do not edit any files.
- Only analyze the project and suggest one refactoring candidate.
- Choose a different code section than my non-AI refactoring example.
- Do not choose only Rename Function or Rename Variable.
- The refactoring should be based on the Martin Fowler refactoring catalog.

Please tell me: 5. Which screenshot should I take before the refactoring?

1. Which class should be refactored?
2. Which method or code section should be refactored?
3. Which Martin Fowler refactoring fits best?
4. Why is this code section suitable for refactoring?

result codex :

1. Class: `Task`
2. Code section: The `toString()` method in `Task.java`, together with its use in `ConsoleUI.showTasks()`.
3. Martin Fowler refactoring: Move Function
4. Why: Formatting console output is a user-interface responsibility. Moving this formatting from `Task` to a private method such as `printTask(Task task)` in `ConsoleUI` would keep `Task` focused on task data while preserving the same output.

2.prompt :

Now please perform the refactoring you selected.

Use the Martin Fowler refactoring that you suggested.

Important:

- Now you are allowed to edit the code.
- Only refactor the selected class and selected method/code section.
- Do not change the behavior of the program.
- Do not add new features.
- Do not add Maven, Gradle, GUI, database, or external libraries.
- Keep the code beginner-friendly.
- The project must still compile with:
`javac -d out src/*.java`
- The program must still run with:
`java -cp out Main`

After editing, please explain shortly:

1. Which file was changed?
2. Which Fowler refactoring was used?
3. What exactly was changed in the code?
4. Why did the behavior stay the same?

5. How can I test that the program still works?

result :

Worked for 32s >

1. Changed [Task.java](C:\Users\zaid\Documents\Smart Task Manger\src\Task.java) and [ConsoleUI.java](C:\Users\zaid\Documents\Smart Task Manger\src\ConsoleUI.java).
2. Used Martin Fowler's **Move Function** refactoring.
3. Removed console formatting from `Task.toString()` and moved it into `ConsoleUI.printTask(Task)`.
4. Behavior stayed the same because all fields, labels, spacing, and the empty-description message remain unchanged.
5. Test with:

○ Add to chat



```
javac -d out src/*.java
java -cp out Main
java -cp out TaskManagerManualTest
```

Compilation succeeded and all six manual tests passed.

4:59 PM



ConsoleUi.java


```

8
9  ✨ private void showTasks(List<Task> tasks){
0  .....if (tasks.isEmpty()){
1  .....    System.out.println("No tasks available. Create a task using menu option 1
2  .....    return;
3  .....}
4
5  .....System.out.println();
6  .....System.out.println("===== TASKS =====");
7  .....for (Task task : tasks){
8  .....    printTask(task);
9  .....    System.out.println("-----");
0  .....}
1  .....}
2
3  .....private void printTask(Task task){
4  .....    String description = task.getDescription().isEmpty()
5  .....    .....? "No description"
6  .....    .....: task.getDescription();
7
8  .....    System.out.println("ID:....." + task.getId());
9  .....    System.out.println("Title:....." + task.getTitle());
0  .....    System.out.println("Description: " + description);
1  .....    System.out.println("Priority:..." + task.getPriority());
2  .....    System.out.println("Status:....." + task.getStatus());
3  .....    System.out.println("Due date:..." + task.getDueDate());
4  .....}
5

```

Tak.javav removed

```

1  import java.time.LocalDate;
2
3  public class Task {
4      private final int id;
5      private final String title;
6      private final String description;
7      private final Priority priority;
8      private TaskStatus status;
9      private final LocalDate dueDate;
10
11     public Task(int id, String title, String description, Priority priority, LocalDate dueDate) {
12         this.id = id;
13         this.title = title;
14         this.description = description;
15         this.priority = priority;
16         this.status = TaskStatus.NEW;
17         this.dueDate = dueDate;
18     }
19
20     public int getId() {
21         return id;
22     }
23
24     public String getTitle() {
25         return title;
26     }
27
28     public String getDescription() {
29         return description;
30     }
31
32     public Priority getPriority() {
33         return priority;
34     }
35
36     public TaskStatus getStatus() {
37         return status;
38     }
39
40     public LocalDate getDueDate() {
41         return dueDate;
42     }
43
44     public void setStatus(TaskStatus status) {
45         this.status = status;
46     }
47
48 }
49

```